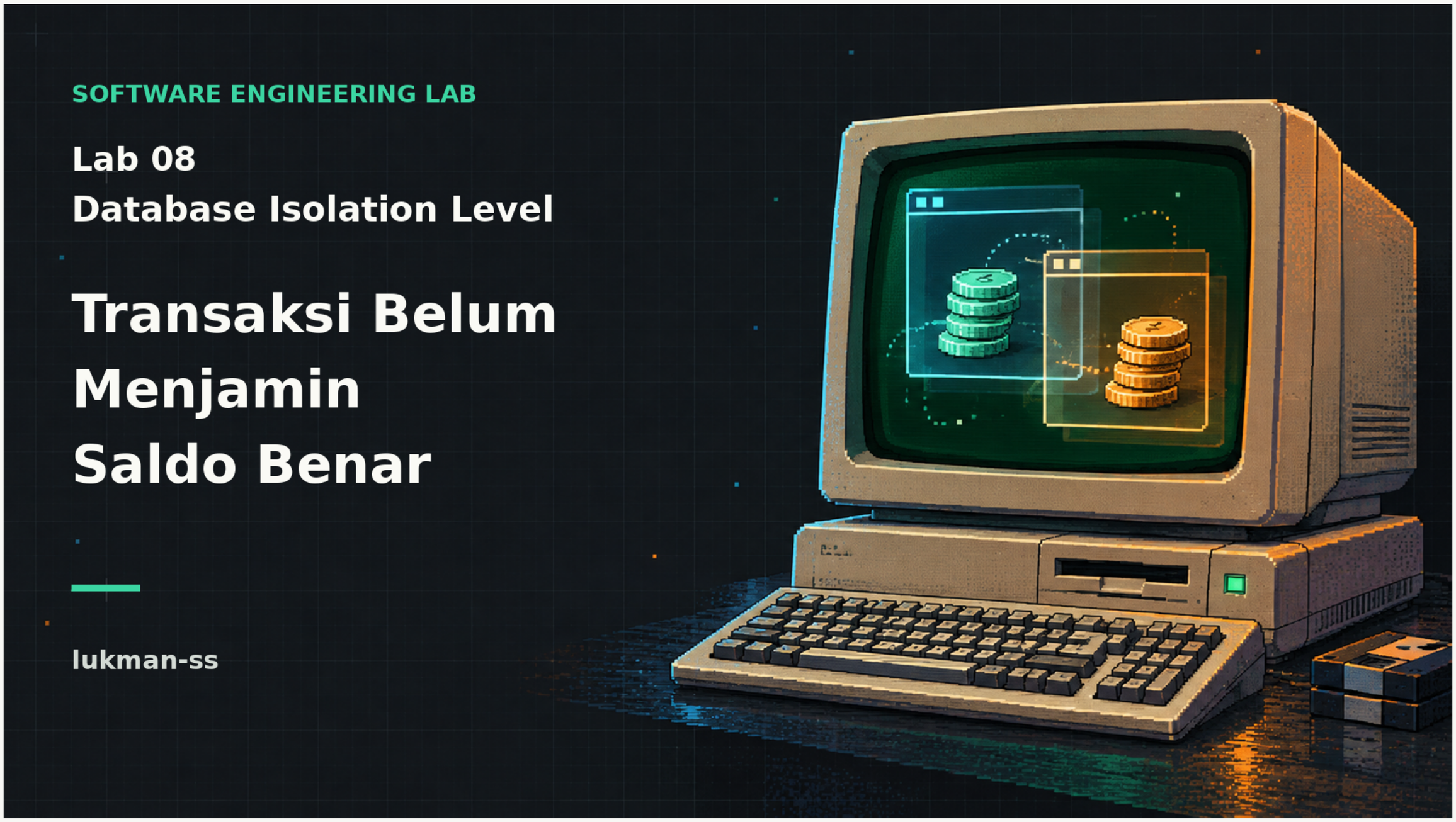
00 / DATABASE ISOLATION LEVEL

Transaksi Belum Menjamin Saldo Benar

Boundary transaksi, snapshot, row locking, dan retry menentukan apakah saldo tetap benar ketika request saling overlap.



Sumber: README.md | Skema / ekspektasi sesuai sumber.

01 / BEGIN / COMMIT BELUM CUKUP

Atomicity mengikat operasi. Isolation mengatur interaksi.

Debit, kredit, dan audit bisa sukses bersama, tetapi keputusan transfer tetap dapat memakai saldo yang sudah kedaluwarsa.

BOUNDARY

Semua query commit atau rollback bersama.

CONCURRENCY

Transaksi lain dapat membaca dan menulis di antara query.

```
TransferNaive menjalankan READ COMMITTED dalam satu transaksi.
```

Transaction boundary tidak memperbarui nilai yang sudah dihitung di aplikasi.

IMPLIKASI PADA LAB

Atomicity tidak mengganti kebutuhan coordination pada keputusan yang bergantung pada state bersama.

Sumber: transfer.go | Skema / ekspektasi sesuai sumber.

02 / TETAPKAN BUSINESS INVARIANT

Saldo positif belum membuktikan uang tetap utuh.

Fixture wallet dasar memakai Alice, Bob, dan Charlie dengan saldo masing-masing Rp1.000.000.

SALDO

Setiap balance >= 0

TOTAL

Jumlah saldo tetap Rp3.000.000

```
balance BIGINT NOT NULL CHECK (balance >= 0)
```

Schema menjaga saldo per baris. Konservasi total perlu diperiksa lintas akun.

IMPLIKASI PADA LAB

Lost update dapat lolos CHECK balance >= 0. Test harus memeriksa jumlah saldo lintas akun.

Sumber: schema.sql | Skema / ekspektasi sesuai sumber.

03 / DUA TRANSAKSI, SATU SALDO

Keduanya membaca Alice = Rp1.000.000.

Transfer A mengirim Rp800.000 ke Bob. Transfer B mengirim Rp800.000 ke Charlie. Keduanya lolos validasi dari bacaan awal yang sama.

TX A

READ Alice 1.000.000; READ Bob 1.000.000

TX B

READ Alice 1.000.000; READ Charlie 1.000.000

```
READ A + READ B -> release barrier -> UPDATE -> COMMIT
```

Test menahan kedua transaksi setelah pembacaan, sebelum penulisan.

URUTAN EKSEKUSI / SKEMA SESUAI SUMBER

READ A

Alice 1.000.000

READ B

Alice 1.000.000

RELEASE

Keduanya menulis

Sumber: transfer_test.go | Skema / ekspektasi sesuai sumber.

04 / LOST UPDATE PADA POLA NAIVE

Nilai lama dihitung, lalu ditulis sebagai nilai absolut.

Kedua transaksi menghitung saldo Alice menjadi Rp200.000. Penulisan kedua menimpa nilai yang sama, walaupun kredit masuk ke dua penerima berbeda.

ALICE

200.000

BOB + CHARLIE

1.800.000 + 1.800.000

```
// 4. Overwrite sender balance with calculated value (Lost Update
vulnerability)
_, err = tx.ExecContext(ctx, "UPDATE isolation_accounts SET balance = $1
WHERE id =
$2", fromBalance-amount, fromID)
if err != nil {
return fmt.Errorf("update from balance: %w", err)
}
```

Risikonya ada pada read-check-calculate-write. Bukan semua query READ COMMITTED.

Sumber: transfer.go | Cuplikan dipersingkat.

05 / APA YANG DIUJI OLEH NAIVE TEST

Total yang diharapkan test: Rp3.800.000.

TestNaiveTransfer_LostUpdate memeriksa saldo akhir Alice, Bob, dan Charlie, lalu menegaskan bahwa total uang menjadi 3.800.000.

CHECK ROW

Semua saldo masih non-negatif.

CHECK SYSTEM

Total bertambah Rp800.000.

```
TestNaiveTransfer_LostUpdate
```

Angka ini adalah assertion sumber, bukan laporan test yang dijalankan di sesi ini.

IMPLIKASI PADA LAB

Channel barrier berada setelah pembacaan. Penulisan baru dilepas ketika kedua transaksi siap.

Sumber: transfer_test.go | Skema / ekspektasi sesuai sumber.

06 / ISOLATION = VISIBILITAS

Transaksi yang sama dapat melihat state berbeda.

PostgreSQL memakai MVCC: pembaca melihat versi data sesuai snapshot. Ordinary SELECT tidak mengambil row lock FOR UPDATE.

SNAPSHOT

Versi mana yang terlihat saat membaca?

LOCK

Siapa yang boleh mengubah row sebelum commit?

```
READ visibility != exclusive access
```

Pisahkan consistent view, penguncian row, dan hasil setara serial execution.

IMPLIKASI PADA LAB

Ordinary SELECT menggunakan versi snapshot. FOR UPDATE menambah koordinasi untuk jalur perubahan row.

Sumber: README.md | Skema / ekspektasi sesuai sumber.

07 / STANDAR SQL VS POSTGRESQL

Tabel standar adalah batas minimum.

Jangan menerapkan tabel anomaly SQL sebagai deskripsi identik untuk semua engine. Lab ini memakai perilaku PostgreSQL.

READ UNCOMMITTED

Standar: dirty read diizinkan. PostgreSQL: setara READ COMMITTED.

REPEATABLE READ

Standar: phantom dapat terjadi. PostgreSQL: phantom klasik dicegah.

```
SERIALIZABLE: hasil commit setara suatu serial execution
```

Perilaku engine perlu dibuktikan oleh skenario, bukan dihafal dari nama level.

IMPLIKASI PADA LAB

READ UNCOMMITTED PostgreSQL tidak memperlihatkan saldo Rp500.000 yang belum di-commit dalam dirty-read test.

Sumber: README.md | Skema / ekspektasi sesuai sumber.

08 / READ COMMITTED

Snapshot baru untuk setiap statement.

Commit transaksi lain di antara dua SELECT dapat terlihat pada SELECT berikutnya, meskipun pembaca belum menutup transaksinya.

SELECT 1

Snapshot sebelum commit transaksi penulis.

SELECT 2

Snapshot sesudah commit transaksi penulis.

```
SELECT -> writer COMMIT -> SELECT
```

Tidak membaca data uncommitted. Tetap bisa membaca nilai committed yang berubah.

URUTAN EKSEKUSI / SKEMA SESUAI SUMBER

SELECT 1	COMMIT B	SELECT 2
Snapshot lama	State baru sah	Snapshot baru

Sumber: read_committed_test.go | Skema / ekspektasi sesuai sumber.

09 / NON-REPEATABLE READ

Row sama, nilai berbeda.

Test pembaca mengambil saldo Alice. Penulis menambah Rp500.000 lalu commit. Pembaca mengambil saldo lagi di transaksi yang sama.

BACA PERTAMA

1.000.000

BACA KEDUA

1.500.000

```
TestReadCommitted_NonRepeatableRead
```

Assertion membandingkan dua bacaan, bukan hanya keberhasilan query.

URUTAN EKSEKUSI / SKEMA SESUAI SUMBER

READ A

1.000.000

COMMIT B

Tambah 500.000

READ A

1.500.000

Sumber: read_committed_test.go | Skema / ekspektasi sesuai sumber.

10 / PHANTOM READ

Predicate sama, jumlah row berubah.

Pembaca menghitung invoice PAID. Penulis menyisipkan invoice PAID ketiga dan commit sebelum pembacaan berikutnya.

READ COMMITTED

COUNT pertama 2; COUNT kedua 3.

REPEATABLE READ

COUNT pertama 2; COUNT kedua 2.

```
SELECT COUNT(*) FROM isolation_invoices WHERE status = 'PAID'
```

Kedua hasil adalah ekspektasi test PostgreSQL dalam repository.

URUTAN EKSEKUSI / SKEMA SESUAI SUMBER

COUNT A	INSERT B	COUNT A
2 PAID	Commit invoice	RC: 3 / RR: 2

Sumber: phantom_read_test.go | Skema / ekspektasi sesuai sumber.

11 / REPEATABLE READ

Satu snapshot untuk pembacaan berulang.

Snapshot diperoleh pada pembacaan pertama. Commit penulis setelah itu tidak mengubah hasil ordinary SELECT dalam snapshot pembaca.

PENULIS

Menambah saldo dan commit.

PEMBACA

Tetap membaca 1.000.000 dua kali.

```
TestRepeatableRead_SnapshotIsolation
```

Consistent snapshot tidak sama dengan hak eksklusif mengubah semua data.

URUTAN EKSEKUSI / SKEMA SESUAI SUMBER

READ A

Snapshot dibuat

COMMIT B

Saldo berubah

READ A

Snapshot tetap

Sumber: repeatable_read_test.go | Skema / ekspektasi sesuai sumber.

12 / CONCURRENT UPDATE CONFLICT

Snapshot lama tidak boleh menimpa update baru.

Dua transaksi REPEATABLE READ membaca row yang sama, lalu mencoba mengurangi Rp800.000. Test mengharapkan satu sukses dan satu serialization failure.

COMMIT

Satu transaksi berhasil.

ABORT

Transaksi yang berbenturan mendapat 40001.

```
TestRepeatableRead_ConcurrentUpdate_SerializationFailure
```

Aplikasi harus menangani conflict; memilih isolation lebih tinggi saja belum lengkap.

URUTAN EKSEKUSI / SKEMA SESUAI SUMBER

READ A+B	COMMIT A	UPDATE B
Snapshot overlap	Update berhasil	Abort 40001

Sumber: repeatable_read_test.go | Skema / ekspektasi sesuai sumber.

13 / SNAPSHOT VS CORRECTNESS

Bacaan konsisten belum menjamin setiap invariant.

README membahas serialization anomaly seperti write skew. Test yang tersedia mereproduksi benturan update row yang sama, bukan eksperimen write skew.

BUKTI LAB

Snapshot stabil dan concurrent update failure.

BATAS BUKTI

Tidak ada reproduksi write skew yang ditunjukkan.

```
Invariant -> read/write set -> strategy
```

Pisahkan konsep dalam README dari perilaku yang benar-benar diuji.

IMPLIKASI PADA LAB

Higher isolation tetap memerlukan strategi abort. Jangan menyebut write skew sebagai hasil eksperimen lab.

Sumber: README.md | Skema / ekspektasi sesuai sumber.

14 / SELECT ... FOR UPDATE

Baca saldo sambil memegang row lock.

Transfer lain yang meminta lock atau mengubah row yang sama menunggu sampai pemegang lock commit atau rollback. Ordinary SELECT tetap memakai MVCC.

LOCK HOLDER

Membaca, memvalidasi, lalu menulis dalam transaksi.

WAITER

Melanjutkan setelah lock dilepas.

```
SELECT balance FROM isolation_accounts WHERE id = $1 FOR UPDATE
```

Waktu tunggu mengikuti contention dan durasi transaksi; bukan latency deterministik.

URUTAN EKSEKUSI / SKEMA SESUAI SUMBER

LOCK A	WAIT B	COMMIT A
Read + validate	Row belum bebas	B dapat lock

Sumber: transfer.go | Skema / ekspektasi sesuai sumber.

15 / TRANSFER DENGAN ROW LOCKING

Lock kedua akun sebelum validasi pengirim.

TransferWithLock memakai READ COMMITTED, mengunci dua akun, menentukan saldo pengirim, lalu memeriksa kecukupan dana.

VALIDASI

Saldo dibaca setelah memperoleh lock.

WRITE

Debit, kredit, dan audit berada dalam transaksi yang sama.

```
firstID, secondID := DeterministicLockOrder(fromID, toID)

// Lock both accounts deterministically
var b1, b2 int64
err = tx.QueryRowContext(ctx, "SELECT balance FROM isolation_accounts WHERE id = $1
FOR UPDATE", firstID).Scan(&b1)
if err != nil {
return fmt.Errorf("lock account %d: %w", firstID, err)
}
err = tx.QueryRowContext(ctx, "SELECT balance FROM isolation_accounts WHERE id = $1
FOR UPDATE", secondID).Scan(&b2)
if err != nil {
return fmt.Errorf("lock account %d: %w", secondID, err)
}
```

Transfer berikutnya mengevaluasi saldo setelah transfer sebelumnya selesai.

Sumber: transfer.go | Cuplikan dipersingkat.

16 / DETERMINISTIC LOCK ORDERING

A -> B dan B -> A memakai urutan lock sama.

Arah transfer tidak menentukan urutan lock. Helper selalu memilih ID lebih kecil dahulu, lalu ID lebih besar.

1 -> 2

LOCK 1; LOCK 2

2 -> 1

LOCK 1; LOCK 2

```
func DeterministicLockOrder(id1, id2 int) (firstID, secondID int) {
if id1 < id2 {
return id1, id2
}
return id2, id1
}
```

Menghindari circular wait pada pola dua akun ini; bukan jaminan bebas semua deadlock.

Sumber: isolation.go | Cuplikan dipersingkat.

17 / SERIALIZABLE DAN SSI

Concurrent execution, hasil setara serial.

PostgreSQL memakai Serializable Snapshot Isolation. Transaksi masih berjalan bersamaan; conflict yang tidak dapat diterima bisa berakhir dengan abort.

SERIAL EQUIVALENCE

Hasil commit setara suatu urutan serial.

CONFLICT HANDLING

Aplikasi siap memulai ulang transaksi yang gagal.

```
TestSerializable_ConcurrentUpdate_SerializationFailure
```

Lab menguji same-row update conflict. Tidak memakai global lock seluruh database.

URUTAN EKSEKUSI / SKEMA SESUAI SUMBER

EXECUTE

Transaksi overlap

CONFLICT

Abort bila perlu

COMMIT

Serial equivalent

Sumber: serializable_test.go | Skema / ekspektasi sesuai sumber.

18 / SQLSTATE 40001 VS 40P01

Dua alasan berbeda untuk mencoba transaksi baru.

WrapTxError mempertahankan original error chain sambil menambahkan sentinel agar errors.Is dan errors.As tetap bekerja.

40001

Serialization failure.

40P01

Deadlock detected.

```
func IsRetryableTxError(err error) bool {
if err == nil {
return false
}
if errors.Is(err, ErrSerializationFailure) || errors.Is(err, ErrDeadlockDetected) {
return true
}
var pqErr *pq.Error
if errors.As(err, &pqErr) {
return pqErr.Code == "40001" || pqErr.Code == "40P01"
}
return false
}
```

Constraint violation dan insufficient funds tidak ikut dianggap retryable.

Sumber: transfer.go | Cuplikan dipersingkat.

19 / BOUNDED RETRY

Ulangi seluruh transaksi, dengan batas.

Wrapper memanggil TransferSerializable lagi: BeginTx, pembacaan, validasi, debit, kredit, audit, dan commit dimulai ulang.

ATTEMPTS

maxAttempts termasuk percobaan pertama; <= 0 ditolak.

BACKOFF

Default 10 ms; exponential cap 1 s; full jitter.

```
func (r *PostgresWalletRepo) TransferSerializableWithRetryPolicy(ctx
context.Context,
fromID, toID int, amount int64, maxAttempts int, policy RetryPolicy) error {
repo := r
return RetryTransaction(ctx, maxAttempts, policy, func(ctx context.Context)
error {
return repo.TransferSerializable(ctx, fromID, toID, amount)
})
}
```

Context dicek sebelum attempt dan saat sleep. Habis attempts: ErrMaxRetryExceeded.

Sumber: transfer.go | Cuplikan dipersingkat.

20 / UJI CONCURRENCY DAN INVARIANT

Orkestrasi overlap, lalu periksa state akhir.

Naive test memakai barrier setelah kedua read. Stress test melepas 100 goroutine melalui start gate dan memeriksa saldo serta total.

FIXTURE DASAR

3 akun x 1.000.000 = 3.000.000

FIXTURE STRESS

3 akun x 50.000 = 150.000; transfer 1.000

```
TestHighContention_100ConcurrentTransfers
```

Start gate memulai beban bersama; tidak menjamin setiap query overlap persis sama.

URUTAN EKSEKUSI / SKEMA SESUAI SUMBER

START GATE

100 goroutine

TRANSFER

Row locking

ASSERT

Total 150.000

Sumber: transfer_test.go | Skema / ekspektasi sesuai sumber.

21 / PILIH BERDASARKAN INVARIANT

Tidak ada satu level untuk semua workload.

Wallet row tertentu memakai lock eksplisit. Pembacaan konsisten memakai snapshot. Invariant kompleks dapat memerlukan serializable dan retry.

ROW LOCK

Biaya: lock wait; disiplin urutan dan durasi transaksi.

SERIALIZABLE

Biaya: conflict abort; retry terbatas dan dependency tracking.

```
Atomic conditional update disebut README; tidak diimplementasikan di lab.
```

Repository tidak menyediakan benchmark untuk meranking throughput atau latency.

IMPLIKASI PADA LAB

Strategi harus cocok dengan read/write set. Nama domain financial tidak menentukan isolation sendirian.

Sumber: README.md | Skema / ekspektasi sesuai sumber.

22 / PELAJARAN UTAMA

Correctness = invariant + mekanisme + conflict handling.

BEGIN/COMMIT menjaga boundary. Snapshot menjaga view. Row locking mengatur akses. Serializable menjaga serial equivalence. Pengujian memeriksa invariant.

DESAIN

Tentukan state yang harus selalu benar.

VERIFIKASI

Buktikan dengan overlap dan assertion state akhir.

```
Boundary -> visibility -> coordination -> invariant
```

Jangan menyamakan request tanpa error dengan sistem yang benar.

IMPLIKASI PADA LAB

Assertion saldo dan total membantu mendeteksi error yang tidak terlihat dari status commit.

Sumber: README.md | Skema / ekspektasi sesuai sumber.

23 / SUMBER DAN IDENTITAS

Software Engineering Lab / Lukman

Semua fakta teknis berasal dari Lab 08. README, schema, implementasi, dan test dibaca. Test tidak dijalankan dalam sesi pembuatan konten ini.

IMPLEMENTASI

schema.sql, isolation.go, transfer.go

EKSPERIMEN

read_committed, repeatable_read, phantom_read, transfer, serializable, retry tests

```
github.com/lukman-ss/software-engineering-lab
labs/08-database-isolation-level
```

lukman-ss | Database Isolation Level

IMPLIKASI PADA LAB

Buka README untuk konteks; baca implementasi untuk perilaku; baca test untuk cakupan assertion.

Sumber: README.md | Skema / ekspektasi sesuai sumber.